

Ejercicio 1:

Por definición, una función $f(n) = O(g(n))$ si existen constantes positivas c y n_0 tales que:

$$0 \leq f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0.$$

Supongamos que $6n^3 = O(n^2)$. Entonces deben existir constantes $c > 0$ y $n_0 \geq 1$ tales que:

$$6n^3 \leq c \cdot n^2 \text{ para todo } n \geq n_0.$$

Dado que $n \geq n_0 \geq 1$, podemos dividir ambos lados por n^2 sin alterar la relación:

$$6n^3 \leq c \cdot n^2 \rightarrow 6n \leq c \rightarrow n \leq c/6$$

La expresión obtenida afirma que n debe ser menor o igual que un número constante ($c/6$) para *todo* $n_0 \geq 1$. Sin embargo, al ser n una variable que puede crecer arbitrariamente (por ejemplo, eligiendo un valor de n tal que $n > c/6$), la desigualdad deja de cumplirse.

Ejercicio 2:

Para lograr el mejor caso en Quicksort $O(n \log n)$, el pivote elegido en cada paso debe dividir al subarreglo en dos mitades exactamente iguales (o de tamaños muy similares).

Asumiendo una implementación común donde el pivote se elige como el elemento central (del medio) del subarreglo, un array ya ordenado de 15 elementos representa el mejor caso perfecto:

Array = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

Primera partición (Nivel 0):

El algoritmo selecciona el elemento central del array completo, que es el 8 (índice medio). Al particionar, todos los elementos menores que 8 quedan a la izquierda y los mayores a la derecha. El array se divide en dos mitades exactas de 7 elementos cada una:

Izquierda: [1, 2, 3, 4, 5, 6, 7]

Derecha: [9, 10, 11, 12, 13, 14, 15]

Segundas particiones (Nivel 1):

Para el subarreglo izquierdo, el elemento central es el 4. Se divide en dos mitades de 3 elementos: [1, 2, 3] y [5, 6, 7]. Para el subarreglo derecho, el elemento central es el 12. Se divide en dos mitades de 3 elementos: [9, 10, 11] y [13, 14, 15].

Conclusión:

Este proceso simula un árbol de recursión perfectamente balanceado. La altura del árbol de llamadas es $\log n$ y en cada nivel se procesan los n elementos, resultando en la complejidad óptima de $O(n \log n)$.

Ejercicio 3:

1 QuickSort:

Tiempo de ejecución / complejidad $O(n^2)$

En las de Quicksort (que parten en dos subarreglos usando un pivote), cuando todos los elementos son iguales, la estrategia de partición agrupa todos los elementos a un solo lado. Esto destruye la división equitativa y hace que el árbol de recursión se vuelva lineal, resultando en una complejidad cuadrática.

2 Insertion-Sort:

Tiempo de ejecución / complejidad $O(n)$

Insertion-Sort compara cada elemento nuevo con los elementos ya ordenados a su izquierda. Como todos los valores son idénticos, la condición de intercambio nunca se cumple (el elemento actual no es menor que los anteriores). El algoritmo simplemente recorre el arreglo de izquierda a derecha haciendo una sola comparación por cada elemento y sin realizar desplazamientos, logrando un rendimiento lineal.

3 Merge-Sort:

Tiempo de ejecución / complejidad $O(n \log n)$

Merge Sort divide siempre el arreglo exactamente a la mitad, sin importar el contenido o el orden de los elementos. El proceso de división recursiva hasta fragmentos de tamaño 1 y la posterior mezcla (*merge*) se ejecutan de manera rígida e invariable, por lo que su tiempo de ejecución es el mismo tanto si los elementos son distintos como si son todos iguales.

Ejercicio 4:

El enfoque fue primero identificar el elemento central y luego los menores a el (estos se iban guardando en una lista junto a su posición) para culminar el problema haciendo los intercambios correspondientes.

Ejercicio 5

El enfoque fue primero ordenar la lista a través de un algoritmo como mergeSort para que la lista quede ordenada de menor a mayor. Luego se sumaban el primer(elemento izq) y último elemento de la lista(elemento der), si la suma coincide se devuelve true, si no coincide se lleva a cabo dos casos. El primer caso es si la suma es mayor al número deseado se mueve el puntero del elemento de la derecha a un elemento a la izquierda y se verifica la suma. El segundo caso es si la suma fue menor al elemento deseado se mueve el puntero del elemento de la izquierda a un elemento a la derecha. Esto se repite hasta que los elementos se crucen.

Ejercicio 6:

Heap Sort:

HeapSort es un algoritmo de ordenamiento basado en comparaciones que utiliza una estructura de datos llamada montículo binario (*binary heap*), específicamente un *max-heap* (montículo máximo) si queremos ordenar de menor a mayor. Su gran ventaja frente a otros algoritmos como Quicksort es que garantiza un rendimiento óptimo en el peor de los casos.

Ordenes de complejidad	
Mejor caso	$O(n \log n)$
Caso promedio	$O(n \log n)$
Peor caso	$O(n \log n)$

Explicación de los casos:

1. **Construcción del montículo (*Heapify*):** Transformar un arreglo desordenado en un montículo válido toma un tiempo lineal de $O(n)$
2. **Extracción de elementos:** El algoritmo extrae el elemento raíz n veces. Cada extracción requiere mover el último elemento a la raíz y reajustar el árbol (*sift-down*), lo que toma $O(\log n)$ por cada uno de los n elementos. El total de esta fase es $O(n \log n)$.
3. **Por qué el peor caso es $O(n \log n)$:** A diferencia de Quicksort (cuyo peor caso degrada a $O(n^2)$ si el pivote está mal elegido), la estructura de un montículo binario siempre es un árbol completo perfectamente balanceado, asegurando que la altura nunca supere $\log(n)$.

Ejemplo de funcionamiento para un caso promedio:

Array inicial = [4, 10, 3, 5, 1, 8, 2, 9]

Fase 1: Construcción del Max-Heap

Convertimos el arreglo en un montículo máximo (donde el padre es siempre mayor o igual que sus hijos):

Al reorganizar los elementos, el arreglo adopta la estructura de un *max-heap*:

Heap = [10, 9, 8, 5, 1, 3, 2, 4]

(El elemento mayor, 10, queda en la raíz, que corresponde a la primera posición del array).

Fase 2: Extracción y Ordenamiento

1. **Intercambio 1:** Intercambiamos la raíz (10, el mayor) con el último elemento del arreglo (4). El arreglo queda parcialmente ordenado al final: [4, 9, 8, 5, 1, 3, 2, 10].
2. **Reajuste (Heapify):** Reducimos el tamaño del montículo considerado (excluyendo el 10 ya ubicado) y restauramos la propiedad de max-heap con el elemento 4 que quedó en la raíz. La nueva raíz pasa a ser 9: Nuevo heap: [9, 5, 8, 4, 1, 3, 2, 10]
3. **Intercambio 2:** Intercambiamos la nueva raíz (9) con el último elemento activo del montículo (2): Array: [2, 5, 8, 4, 1, 3, 9, 10]
4. **Repetición del proceso:** El algoritmo repite este ciclo de extraer la raíz y reajustar el árbol hasta que todos los elementos quedan ubicados de forma ascendente:

Array final ordenado = [1, 2, 3, 4, 5, 8, 9, 10]

BucketSort:

BucketSort es un algoritmo de ordenamiento por distribución que funciona dividiendo los elementos de un arreglo en un número finito de "cubetas" (*buckets*). Cada cubeta se ordena de forma independiente (habitualmente utilizando otro algoritmo como *InsertionSort* o aplicando el mismo algoritmo de forma recursiva) y, finalmente, se concatenan todas las cubetas en orden para obtener el arreglo completamente ordenado. Es altamente eficiente cuando los datos están distribuidos de manera uniforme en un rango determinado.

Ordenes de complejidad	
Mejor caso	$O(n+k)$
Caso promedio	$O(n+k)$
Peor caso	$O(n^2)$

Mejor caso: $O(n + k)$ (donde n es el número de elementos y k es el número de cubetas. Ocurre cuando los elementos se distribuyen de forma perfectamente uniforme entre las cubetas (un elemento por cubeta, o una cantidad muy pequeña y constante) y el ordenamiento interno de cada cubeta toma tiempo lineal.

Caso promedio: $O(n + k)$. Si los datos están distribuidos de manera aleatoria o uniforme, la cantidad de elementos por cubeta es baja y manejable, lo que permite que el ordenamiento interno de cada cubeta sea muy rápido, resultando en un tiempo lineal en promedio.

Peor caso: $O(n^2)$. Ocurre cuando todos los elementos del arreglo caen dentro de la misma cubeta. Esto destruye la ventaja de la distribución y obliga al algoritmo interno de la cubeta (por ejemplo, *InsertionSort*) a procesar todos los elementos juntos, degradando su rendimiento a una complejidad cuadrática.

Ejemplo de funcionamiento para un caso promedio:

Array inicial = [0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51, 0.38]

Fase 1: Creación y Distribución en Cubetas

1. Creamos $k = 5$ cubetas vacías que abarcan rangos iguales:
 - Cubeta 0: [0.0, 0.2)
 - Cubeta 1: [0.2, 0.4)
 - Cubeta 2: [0.4, 0.6)
 - Cubeta 3: [0.6, 0.8)
 - Cubeta 4: [0.8, 1.0)
2. Distribuimos cada elemento del arreglo en su cubeta correspondiente según su valor:
 - Cubeta 0: []
 - Cubeta 1: [0.32, 0.23, 0.25, 0.38]
 - Cubeta 2: [0.42, 0.52, 0.47, 0.51]
 - Cubeta 3: []
 - Cubeta 4: []

Fase 2: Ordenamiento Interno

Ordenamos los elementos dentro de cada cubeta de manera individual (por ejemplo, usando *InsertionSort*):

- Cubeta 1 ordenada: [0.23, 0.25, 0.32, 0.38]
- Cubeta 2 ordenada: [0.42, 0.47, 0.51, 0.52]

Fase 3: Concatenación

Unimos las cubetas de izquierda a derecha en orden secuencial para reconstruir el arreglo definitivo:

Array final ordenado = [0.23, 0.25, 0.32, 0.38, 0.42, 0.47, 0.51, 0.52]

RadixSort:

RadixSort es un algoritmo de ordenamiento no basado en comparaciones. En su lugar, ordena los datos agrupando las claves por sus dígitos individuales (o por posiciones de caracteres, si son palabras). Funciona procesando los números dígito por dígito, desde el dígito menos significativo (LSD) hasta el más significativo (MSD), utilizando un algoritmo de ordenamiento auxiliar estable (como *Counting Sort*) en cada pasada.

Ordenes de complejidad	
Mejor Caso	$O(k \cdot n)$
Caso Promedio	$O(k \cdot n)$
Peor Caso	$O(k \cdot n)$

El tiempo de ejecución depende estrictamente del número de elementos (n) y de la cantidad de dígitos o longitud de la clave (k). Como el algoritmo siempre recorre todos los dígitos de cada número de manera invariable (sin importar cuán desordenados o distribuidos estén inicialmente), los escenarios mejor, promedio y peor comparten exactamente la misma complejidad temporal. Si k se considera una constante pequeña (por ejemplo, números enteros de hasta 8 dígitos), el rendimiento se comporta como $O(n)$ (tiempo lineal).

Ejemplo de funcionamiento para un caso promedio:

Array inicial = [329, 457, 657, 839, 436, 720, 355]

Fase 1: Ordenamiento por el dígito menos significativo (Unidades)

Agrupamos los números según su última cifra (dígito de las unidades):

Dígitos: 9, 7, 7, 9, 6, 0, 5

Al ordenarlos de forma estable según las unidades, el arreglo queda así:

Array parcial 1 = [720, 355, 436, 457, 657, 329, 839]

Fase 2: Ordenamiento por el dígito intermedio (Decenas)

Tomamos el resultado anterior y agrupamos los números según su cifra central (decenas):

Dígitos: 2, 5, 3, 5, 5, 2, 3

Al aplicar un ordenamiento estable basado en las decenas:

Array parcial 2 = [720, 329, 436, 839, 355, 457, 657]

Fase 3: Ordenamiento por el dígito más significativo (Centenas)

Finalmente, ordenamos los elementos basándonos en la primera cifra (centenas):

Dígitos: 7, 3, 4, 8, 3, 4, 6

Tras esta última pasada, el arreglo queda completamente ordenado:

Array final ordenado = [329, 355, 436, 457, 657, 720, 839]

Ejercicio 7:

a) $T(n) = 2T\left(\frac{n}{2}\right) + n^4$

$a = 2$ $b = 2$ $S(n) = n^4$

$T(n) = O(n^{\log_2 2^2}) = \Omega(n^{1+\epsilon}) = n^4$ para $\epsilon = 3$
 $= O(n^4)$

Aplica el caso 3:

$$\begin{aligned} 1. \quad S\left(\frac{n}{b}\right) &\leq c \cdot S(n) \\ 2. \quad \left(\frac{n}{2}\right)^4 &\leq c \cdot n^4 \\ 2 \cdot \frac{n^4}{16} &\leq c \cdot n^4 \\ \frac{n^4}{8} &\leq c \cdot n^4 \end{aligned}$$

Cumple la condición de regularidad, Osea: $T(n) = \Theta(S(n))$

$T(n) = \Theta(n^4)$

b) $T(n) = 25T\left(\frac{7n}{10}\right) + n$

$a = 25$ $b = \frac{10}{7}$ $S(n) = n$ $n^{\log_{\frac{10}{7}} 25} \approx n^{1.94}$

Comparamos n con $n^{1.94}$. Como $1 < 1.94$ estamos en el caso 1, $S(n)$ crece más lento que $n^{\log_{\frac{10}{7}} 25}$, por lo tanto:

$T(n) = \Theta(n^{1.94})$

c) $T(n) = 16T\left(\frac{n}{4}\right) + n^2$

$a = 16$ $b = 4$ $S(n) = n^2$ $n^{\log_4 16} = n^{\log_2 4} = n^2$

Comparamos $S(n)$ con $n^{\log_4 16}$ y vemos que $S(n) = n^{\log_4 16} \rightarrow n^2 = n^2$, por lo tanto estamos en el segundo caso:

$T(n) = \Theta(n^{\log_4 16} \log n)$

$T(n) = \Theta(n^2 \log n)$

Método Maestro Simplificado: $T(n) = aT(n/b) + n^c$

d) $T(n) = 7T\left(\frac{n}{3}\right) + n^2$

$a = 7 \quad b = 3 \quad c = 2$

Comparamos $\log_b a$ con c : $\log_3 7 = 1.77$

Como $\log_3 7 < 2$, estamos en el caso 1:

$$T(n) = \Theta(n^2)$$

e) $T(n) = 7T\left(\frac{n}{2}\right) + n^2$

$a = 7 \quad b = 2 \quad c = 2$

Comparamos $\log_b a$ con c : $\log_2 7 = 2.81$

Como $\log_2 7 > 2$, estamos en el caso 3:

$$T(n) = \Theta(n^{2.81})$$

f) $T(n) = 2T\left(\frac{n}{4}\right) + n^{1/2}$

$a = 2 \quad b = 4 \quad c = 1/2$

Comparamos $\log_b a$ con c : $\log_4 2 = 1/2$

Como $\log_4 2 = 1/2$, estamos en el caso 2:

$$T(n) = \Theta(n^{1/2} \log n)$$

Orden de crecimiento: $s: \Theta(n^{1/2} \log n)$

$b: \Theta(n^{1.4})$

$c: \Theta(n^2 \log n)$

$d: (n^2)$

$e: (n^{2.81})$

$a: (n^4)$